

Guía de despliegue

WebLab en cPanel

Sistema de Gestión de Diagnósticos Clínicos Veterinarios
Laravel 13 + Inertia/React sobre hosting compartido sin acceso SSH

Servidor destino	webapp.cefppj.org.mx
Cuenta cPanel	cefpotgn654
Panel	cPanel & WHM v134.0.56 sobre CloudLinux v8.10
Repositorio	github.com/danicastillodev/WEBLab
Base de datos	cefpotgn654_weblab
Fecha	24 de septiembre de 2026 (2.ª revisión)

1. Resumen: qué se puede hacer en este servidor

Revisé la cuenta de cPanel de **webapp.cefppj.org.mx**. El servidor **sí puede correr WebLab**, pero no de la forma habitual: no hay SSH, ni Terminal, ni Node.js, ni Composer. Eso descarta el flujo clásico de *git pull + composer install + npm run build* en el servidor.

La estrategia viable es **compilar fuera y publicar ya compilado**: GitHub Actions instala dependencias de PHP y compila los assets de Vite, y deja el resultado en una rama `deploy` que ya incluye `vendor/` y `public/build`. cPanel solo clona esa rama y ejecuta un archivo `.cpanel.yml` que copia archivos y corre las migraciones.

El sistema ya está desplegado y funcionando

PHP. La web corre 8.3.33 y el despliegue usa `/opt/alt/php83/usr/bin/php` para la línea de comandos.

Base de datos. `cefpotgn654_weblab`, migrada completa en InnoDB y con los catálogos sembrados.

Despliegue. El circuito entero funciona: push a `main`, GitHub Actions compila, cPanel clona la rama `deploy` y ejecuta `.cpanel.yml`.

Sitio. `/login` responde 200 y renderiza; `/env` responde 403.

Crons. Los tres permanentes dados de alta.

Esta guía pasa entonces de ser un plan a ser **documentación de lo que hay**: para reproducir el montaje en otro servidor, para entender por qué cada pieza está como está, y sobre todo para las tres trampas de abajo, que costaron horas y reaparecerían en cualquier despliegue nuevo.

Los tres puntos que todavía pueden morder

1. El repositorio de GitHub es **privado**, así que cPanel no puede clonarlo sin credenciales. Hace falta un token de acceso (Parte C).

2. `.cpanel.yml` corre `migrate` pero **no** `db:seed`: razas, pruebas y funciones zootécnicas se quedan vacías si no se siembran aparte (Parte E).

3. Tres trampas del servidor que ya están resueltas pero conviene conocer, porque reaparecen a la mínima: el `public_path()` mal deducido (sección 3.1), el PHP de línea de comandos que no sigue al de la web (B.1) y el motor MyISAM del servidor, que descarta las claves foráneas en silencio (E.2).

2. Inventario del servidor

2.1 Datos de la cuenta

Concepto	Valor
Usuario cPanel	cefpotgn654

Concepto	Valor
Directorio home	/home/cefpotgn654
Dominio principal	cefppj.org.mx
IP compartida	66.102.128.157
Acceso al panel	https://webapp.cefppj.org.mx:2083
Sistema	CloudLinux v8.10.0 — cPanel & WHM v134.0.56
Disco	42.17 GB de 97.66 GB usados (43%)
Inodos	86,951 de 300,000 usados (29%)
Base de datos	91.39 MB de 55.57 GB

2.2 Herramientas disponibles

Herramienta	Estado	Para qué sirve en este despliegue
Git Version Control	DISPONIBLE	Clonar la rama <code>deploy</code> y ejecutar <code>.cpanel.yml</code> . Es el motor del despliegue.
Select PHP Version	DISPONIBLE	Ofrece 8.3, 8.4 y 8.5. Hoy está en 7.0. Aquí se activan también las extensiones.
Cron Jobs	DISPONIBLE	Scheduler de Laravel y procesamiento de colas. Admite frecuencia de un minuto.
MySQL / phpMyAdmin	DISPONIBLE	Crear la base nueva, el usuario, y revisar datos.
Manage API Tokens	DISPONIBLE	Token para el MCP <code>weblab-deploy</code> , que permite desplegar desde Claude Code.
File Manager / FTP	DISPONIBLE	Crear el <code>.env</code> , leer logs, plan B de subida manual.
SSL/TLS + AutoSSL	DISPONIBLE	Certificado activo. Falta encender Force HTTPS en el subdominio.
Backup / Backup Wizard	DISPONIBLE	Respaldo previo obligatorio antes de tocar el PHP global.
WHM (revendedor)	PARCIAL	Hay acceso a WHM pero limitado: Account Functions, Packages, DNS y CloudLinux Manager. No incluye Manage Shell Access.

2.3 Lo que NO está disponible

Ausencia	Consecuencia directa
SSH / Terminal	No se puede correr artisan, composer ni npm a mano. Todo comando pasa por <code>.cpanel.yml</code> o por un cron.
Setup Node.js App	Vite no puede compilarse en el servidor. Los assets deben llegar ya compilados.
Composer	Las dependencias PHP deben llegar ya instaladas en <code>vendor/</code> .
Aislamiento por dominio	cPanel avisa: <i>"Site isolation has been denied by your server administrator"</i> . Los 4 dominios comparten una sola versión de PHP.

Intenté abrir *Manage Shell Access* desde WHM y el servidor respondió 403 Forbidden: el proveedor no delegó ese privilegio. Habilitar SSH requiere pedírselo al hosting.

2.4 Dominios de la cuenta

Dominio	Document root	PHP	Contenido
cefppj.org.mx (<i>principal</i>)	/public_html	7.0 global	Sitio en producción
dms.cefppj.org.mx	/dms.cefppj.org.mx	7.0 global	Sitio en producción
siepesa.cefppj.org.mx	/siepesa.cefppj.org.mx	7.0 global	Sitio en producción
webapp.cefppj.org.mx	/webapp.cefppj.org.mx	7.0 global	Vacío (solo un index.html de relleno)

El subdominio destino tiene su propia raíz, separada de `public_html`. Eso es bueno: WebLab no comparte carpeta con el sitio principal.

2.5 Bases de datos de la cuenta

Base	Tamaño	Usuario con permisos	Observación
cefpotgn654_weblab	nueva	<i>por confirmar</i>	El destino de WebLab.
cefpotgn654_netlab	90.24 MB	cefpotgn654_user_web	No tocar. Es el NETLab anterior.
cefpotgn654_qr	204 KB	cefpotgn654_db_admin	Ajena al proyecto
cefpotgn654_web	976 KB	cefpotgn654_user_web	Ajena al proyecto

Confirma el usuario de la base nueva

Crear la base no basta: Laravel necesita un **usuario de MySQL asignado a cefpotgn654_weblab con ALL PRIVILEGES**, porque las migraciones crean y alteran tablas. Se hace en **cPanel** → **Databases** → **MySQL Databases** → **Add User To Database**.

Sobre la base de datos heredada

El diagrama de clases del NETLab original documenta el mismo dominio (HistoriaClinica, Propietario, Especie, Raza...). Si el Laravel nuevo corriera migrate --force contra cefpotgn654_netlab, chocaría con las tablas existentes o las sobrescribiría. Se crea una base nueva y, si hace falta conservar los datos históricos, se migran después con un proceso aparte.

3. Cómo va a funcionar el despliegue

Tres piezas, ninguna de las cuales necesita SSH:

Paso	Dónde ocurre	Qué hace
1. Escribir código	Tu Mac	Se hace <code>git push</code> a la rama <code>main</code> .
2. Compilar	GitHub Actions	<code>composer install --no-dev</code> y <code>npm run build</code> . Publica todo, con <code>vendor/</code> y <code>public/build</code> , en la rama <code>deploy</code> .
3. Publicar	cPanel	Git Version Control hace <i>pull</i> de <code>deploy</code> y ejecuta <code>.cpanel.yml</code> : copia archivos, corre <code>migrate</code> y cachea configuración.

3.1 Dónde queda cada cosa en el servidor

```

/home/cefpotgn654/
|
+-- repositories/WEBlab/      <- clon de la rama "deploy" (Git Version Control)
|
+-- weblab/                  <- LA APLICACION, fuera de la web
|   +-- app/ bootstrap/ config/ database/ routes/ vendor/
|   +-- .env                  <- se crea a mano, NUNCA se versiona
|   +-- storage/              <- logs, cache, sesiones (sobrevive al deploy)
|
+-- webapp.cefppj.org.mx/    <- DOCUMENT ROOT (lo publico)
    +-- index.php             <- copia de deploy/cpanel-index.php
    +-- .htaccess             <- viene de public/
    +-- build/                <- assets compilados por Vite
    +-- storage -> ../weblab/storage/app/public

```

El código y el `.env` quedan fuera del document root: desde internet no hay forma de descargarlos aunque PHP deje de interpretarse.

Por qué esta separación importa

En hosting compartido la raíz del dominio se sirve tal cual. Si el proyecto completo viviera dentro de `webapp.cefppj.org.mx/`, cualquiera podría pedir `.env` y llevarse las credenciales de la base de datos. El archivo `deploy/cpanel-index.php` existe justamente para cargar Laravel desde `../weblab`. Ya está comprobado que funciona: pedir `.env` por HTTP devuelve 403.

El detalle que cuesta un 500

Separar la carpeta pública tiene una consecuencia que no es evidente. Laravel deduce `public_path()` como `base_path('public')`, es decir `~/weblab/public` — que **no existe**, porque `.cpanel.yml` excluye `/public` del `rsync` a propósito. Entonces `@vite` busca el manifest en una ruta inexistente, lanza `ManifestNotFoundException` y toda página que renderice una vista responde 500.

Por eso el front controller le dice explícitamente a Laravel dónde está la carpeta pública, antes de atender la petición:

```
$app = require_once $APP_BASE.'/bootstrap/app.php';  
  
$app->usePublicPath(__DIR__);  
  
$app->handleRequest(Request::capture());
```

Cómo se reconoce este fallo

Es engañoso, porque **no todo responde 500**. Una ruta inexistente sigue devolviendo un 404 correcto, y los archivos estáticos como `/build/manifest.json` se sirven con 200, porque nada de eso pasa por `@vite`. Parece que el framework está roto a medias cuando en realidad arranca perfecto y solo muere al renderizar. Si ves ese patrón —404 bien, estáticos bien, páginas reales en 500— mira `public_path()` antes que la base de datos.

4. Parte A — Preparar el servidor

Esta parte ya está aplicada

PHP está en **8.3.33** y `cefpotgn654_weblab` ya existe. Se conserva como referencia de lo hecho y por las dos comprobaciones pendientes: **A.1** (que los otros tres sitios sigan vivos) y el usuario de base de datos de la sección 2.5. Si nada de eso está en duda, salta a la **Parte C**.

A.1 Comprobar qué rompe el cambio de PHP (pendiente de verificar)

Los cuatro dominios comparten la versión de PHP, así que el salto a 8.3.33 alcanzó también a `cefppj.org.mx`, `dms` y `siepesa`. Si todavía no los has abierto desde el cambio, hazlo ahora: es lo único del cambio que no se puede dar por bueno sin mirarlo.

- Abre las tres URLs y compáralas con cómo se veían antes. Un error 500 o una página en blanco es la señal típica de incompatibilidad.
- Si alguna falló, mira qué corre en **File Manager**: `/public_html`, `/dms.cefppj.org.mx` y `/siepesa.cefppj.org.mx`. Con `wp-config.php` es WordPress, que actualizado aguanta 8.3 sin problema; el código a medida escrito para PHP 5/7 es el que suele romperse, por `mysql_*`, constructores al estilo antiguo o `each()`.
- Si algo se rompió, vuelve a 7.0 en *Select PHP Version* y aplica una de las tres salidas de abajo. WebLab puede esperar; los sitios en producción no.

Si los otros sitios no toleran PHP 8.3, tienes tres salidas

Opción	Cómo	Comentario
Handler por <code>.htaccess</code>	Añadir <code>AddHandler application/x-httpd-alt-php83___lsphp .php</code> en la raíz de <code>webapp</code> y dejar el global en 7.0	Es lo primero que conviene probar: no toca a nadie más. Funciona en muchas configuraciones CloudLinux, pero no en todas.
Pedir aislamiento	Solicitar al hosting que habilite <i>site isolation</i> (CageFS + PHP Selector por dominio)	La solución correcta y definitiva. Depende del proveedor.
Cuenta cPanel aparte	Desde tu WHM de revendedor, crear una cuenta nueva solo para WebLab	Tienes el privilegio <i>Create a New Account</i> . Aísla del todo, pero implica mover el subdominio y consumir otra cuenta.

A.2 Versión de PHP y extensiones (hecho: 8.3.33)

La cuenta ya está en **8.3.33**, en **cPanel** → **Software** → **Select PHP Version** → **Account Global Settings**. Lo que conviene repasar es la pestaña **Extensions**, porque el selector de CloudLinux no arrastra necesariamente las mismas extensiones que tenía 7.0. Deben estar marcadas:

bcmath	ctype	curl	dom	fileinfo
filter	gd	iconv	intl	json
mbstring	openssl	pcre	pdo	pdo_mysql
session	tokenizer	xml	xmlwriter	zip

Falta pdo_mysql y la aplicación no arranca; falta intl o mbstring y se rompe a medias, con errores que cuesta rastrear.

En la pestaña **Options**, sube memory_limit a 256M, max_execution_time a 120 y upload_max_filesize / post_max_size a un valor acorde a los archivos que suba el laboratorio.

A.3 Base de datos (hecha: cefpotgn654_weblab)

La base ya existe y está separada del cefpotgn654_netlab heredado, que conserva 90 MB del sistema anterior y no debe tocarse. Lo único pendiente es el **usuario de MySQL**: si no lo creaste junto con la base, ve a **cPanel** → **Databases** → **MySQL Databases**, créalo en *Add New User* con el *Password Generator*, y asígnalo en *Add User To Database* con **ALL PRIVILEGES**. Guarda esa contraseña: la necesitas en la Parte D.

A.4 Crear el token de API de cPanel

Este token permite que el servidor MCP weblab-deploy del repositorio dispare despliegues desde Claude Code, sin abrir el navegador.

- **cPanel** → **Security** → **Manage API Tokens** → **Create**.
- Nombre: weblab-deploy.
- Copia el token en cuanto aparezca: cPanel no vuelve a mostrarlo.

A.5 Forzar HTTPS

En **cPanel** → **Domains** → **Domains**, en la fila de **webapp.cefppj.org.mx**, activa **Force HTTPS Redirect**. Hoy está en *Off*. Laravel genera URLs absolutas y las sesiones se comportan mal si el sitio responde por HTTP y HTTPS a la vez.

5. Parte B — Preparar el repositorio

Esta parte ya está aplicada y verificada

Los 170 archivos pendientes están commiteados y subidos a main, GitHub Actions compiló en verde y la rama deploy existe. Queda como registro de qué se cambió y por qué, y porque la **APP_KEY de B.2 sigue haciendo falta** para la Parte D.

B.1 Las correcciones que llevaba el .cpanel.yml

El archivo publicaba en \$HOME/public_html, que es el **sitio principal**: desplegarlo así habría sobrescrito cefppj.org.mx. Se corrigieron dos cosas, ya presentes en la rama deploy:

```
# El destino, que ahora es el subdominio y no el sitio principal
- export WEB="$HOME/webapp.cefppj.org.mx"

# Y el borrado del index.html de relleno, en el bloque 3
- /bin/rm -f "$WEB/index.html"
```

Lo segundo importa más de lo que parece: el subdominio traía un index.html de relleno, y Apache lo sirve antes que index.php. Sin borrarlo, el despliegue funcionaría y aun así verías una página en blanco.

Cambio 3 — el binario de PHP, que costó caro

/usr/local/bin/php NO es el PHP del sitio

Este fue el fallo que más tiempo costó. En este servidor la web del subdominio corre **8.3.33**, pero /usr/local/bin/php —la ruta que el propio cPanel sugiere en sus ejemplos de cron— la gobierna el selector de CloudLinux, que seguía en **7.0.33**. Lo confirma ~/.cl.selector, donde el único archivo de configuración era alt_php70.cfg.

El síntoma era desconcertante: los archivos se copiaban bien y artisan moría al arrancar, antes de ejecutar nada.

```
PHP Fatal error: Composer detected issues in your platform:
Your Composer dependencies require a PHP version ">= 8.3.0".
You are running 7.0.33.  -> /home/cefpotgn654/webapp/artisan
```

Por eso .cpanel.yml ya no confía en ninguna ruta fija: recorre los binarios conocidos y se queda con el primero que sea **de verdad** 8.3 o superior, comprobando PHP_VERSION_ID en lugar de suponerlo por el nombre del directorio. Si no encuentra ninguno, falla ahí mismo con un mensaje claro en vez de reventar más adelante dentro de artisan.

```
- export PHP=$(for p in /opt/cpanel/ea-php83/root/usr/bin/php \
                        /opt/alt/php83/usr/bin/php \
                        /opt/cpanel/ea-php84/root/usr/bin/php \
                        /opt/alt/php84/usr/bin/php \
                        /usr/local/bin/php; do \
    [ -x "$p" ] && "$p" -r 'if (PHP_VERSION_ID < 80300) exit(1);' 2>/dev/null \
    && { echo "$p"; break; }; done)
- test -n "$PHP" || { echo "ERROR - no hay PHP 8.3+"; exit 1; }
```

En este servidor eligió `/opt/alt/php83/usr/bin/php`. **Esa misma ruta es la que deben usar los crons de la Parte F:** con `/usr/local/bin/php` morirían en silencio cada minuto.

El fallo que se bloqueaba a sí mismo

Cuando artisan moría, PHP escribía un archivo `error_log` en el directorio de trabajo —que es el propio repositorio clonado—. Ese archivo sin rastrear hacía que cPanel se negara a desplegar con “*The system cannot deploy*”. Es decir: **un despliegue fallido impedía el siguiente**, y no había forma de salir del bucle sin borrarlo a mano. El workflow ya añade `error_log` al `.gitignore` de la rama `deploy` para que no vuelva a pasar.

Cómo queda el flujo de archivos

`rsync` copia el código a `~/weblab` excluyendo `.env`, `storage/` y `public/`; luego copia `public/` al document root y coloca `cpanel-index.php` como `index.php`. Las exclusiones son las que hacen que tu configuración y tus logs sobrevivan a cada despliegue.

B.2 Generar la APP_KEY (pendiente)

Sin shell no puedes correr `php artisan key:generate` en el servidor. Genérala en tu Mac y guarda el resultado para la Parte D:

```
php artisan key:generate --show
```

Devuelve algo como `base64:Xk3...=`. Cópialo completo, incluido el prefijo `base64:`.

B.3 El administrador de prueba, fuera de producción

`DatabaseSeeder` creaba un usuario `admin` con la contraseña literal `password` y permisos totales. Ahora solo se siembra cuando `APP_ENV` no es `production`. Por eso el `.env` del servidor debe declarar `APP_ENV=production`: no es decorativo, es lo que cierra esa puerta.

En su lugar, el administrador real se crea con un comando nuevo, pensado para un servidor sin shell — no pregunta nada y se puede lanzar desde un cron:

```
php artisan weblab:crear-admin <usuario> <correo> [--name="..."] [--password=...]
```

Si omities `--password`, genera una al azar y la imprime en su salida. Se usa en la Parte E.

B.4 Estado del repositorio

Qué	Estado
Rama main	Al día, con los 170 archivos
Rama deploy	Creada por GitHub Actions
vendor/ en deploy	Presente
public/build en deploy	Presente: 69 archivos, 928 KB
database/data/municipios.json	Presente: 2,478 municipios
.cpanel.yml publicado	Con el destino ya corregido

6. Parte C — Clonar el repositorio en cPanel

Esta parte ya está aplicada

El repositorio está clonado en cPanel, apuntando a la rama `deploy`. Queda documentada porque el token de GitHub caduca —y cuando lo haga, el despliegue dejará de traer cambios sin decir por qué— y porque el rechazo de C.2 es lo que más tiempo cuesta si se vuelve a tropezar con él.

C.1 Crear un token de acceso en GitHub

WEBLab es un repositorio privado

La API pública de GitHub responde 404 sin autenticación, así que cPanel **no puede clonarlo con la URL a secas**. El clon fallará sin decir gran cosa si se intenta sin credenciales.

- En GitHub: **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
- **Repository access**: solo `danicastillodev/WEBLab`.
- **Permissions** → **Repository permissions** → **Contents: Read-only**. No hace falta nada más; el servidor solo necesita leer.
- Ponle una caducidad y apúntala: cuando expire, el despliegue dejará de traer cambios.

Dónde acaba ese token

Queda en texto plano en el servidor, dentro de `.git-credentials`. Por eso conviene que sea de solo lectura y limitado a este repositorio: si alguien lo obtiene, no puede escribir en el código ni tocar tus otros proyectos.

C.2 Preconfigurar el acceso, antes de clonar

cPanel rechaza las URL con contraseña

Si intentas clonar con `https://usuario:token@github.com/...`, cPanel responde *"The clone URL cannot include a password"* y no te deja continuar. El propio formulario lo avisa a su manera: *"In order to clone private repositories, advanced users should preconfigure access"*. Hay que dejar las credenciales puestas **antes**, y clonar con la URL limpia.

Git lee `~/.git-credentials` cuando el ayudante `store` está activo. Todo se hace desde File Manager, sin shell:

- En **File Manager**, activa **Settings** → **Show Hidden Files (dotfiles)**.
- En `/home/cefpotgn654`, abre `.gitconfig` (o créalo) y **añade al final**, sin borrar lo que ya tenga:

```
[credential]
    helper = store
```

- En la misma carpeta, crea `.git-credentials` con una sola línea. Solo el host: sin ruta y sin `.git` al final.

```
https://danicastillodev:TU_TOKEN@github.com
```

- Selecciona `.git-credentials` → **Permissions** → `0600`. Es un archivo con un token; que no lo lea nadie más.

Alternativa: llave SSH

Es lo que recomienda la documentación de cPanel, pero esta cuenta no tiene *SSH Access* en el panel, así que habría que colocar las llaves a mano por File Manager (privada en `~/ .ssh/id_ed25519` con permisos `0600`, la carpeta en `0700`, un config que la apunte, y la pública como *Deploy Key* de solo lectura en GitHub). Además depende de que el hosting permita salida por el puerto 22. Queda de reserva.

C.3 Crear el repositorio

- **cPanel** → **Files** → **Git Version Control** → **Create**.
- Activa el interruptor **Clone a Repository**.
- **Clone URL**, ahora limpia, sin credenciales:

```
https://github.com/danicastillodev/WEBLab.git
```

- **Repository Path**: `repositories/WEBLab`
- **Repository Name**: `WebLab`
- Pulsa **Create** y espera. El clon inicial tarda, porque la rama `deploy` lleva `vendor/` dentro.

C.4 Apuntar el repositorio a la rama deploy

cPanel clona la rama por defecto (`main`), que **no** trae dependencias compiladas. Hay que cambiarla:

- En la lista de repositorios, pulsa **Manage** sobre `WebLab`.
- Pestaña **Pull or Deploy**. En el selector **Checked-Out Branch**, elige `deploy`.
- Pulsa **Update** y luego **Update from Remote**.

Verificación obligatoria antes de seguir

Abre **File Manager** en `/home/cefpotgn654/repositories/WEBLab` y confirma que existen las carpetas `vendor/` y `public/build/`. Si no están, el checkout se quedó en `main` y el despliegue fallará con *Class not found*.

C.5 Vigilar el consumo de inodos

La cuenta usa 86,951 de 300,000 inodos. Entre el clon en `repositories/` y la copia en `weblab/` tendrás `vendor/` duplicado: calcula unos 40,000 inodos adicionales. Hay margen, pero conviene revisar el contador en la barra lateral de cPanel después del primer despliegue.

7. Parte D — Crear el archivo .env

Este paso va antes del primer despliegue

`.cpanel.yml` ejecuta `artisan migrate --force` al final. Sin `.env`, Laravel no sabe a qué base conectarse y el despliegue termina con error. Crea el archivo primero.

- En **File Manager**, sitúate en `/home/cefpotgn654` y crea la carpeta `web lab` con **+ Folder**.
- Entra en ella, pulsa **+ File** y nómbrala `.env` (con el punto inicial).
- Activa **Settings** → **Show Hidden Files (dotfiles)** si no la ves.
- Selecciónala y pulsa **Edit**. Pega el contenido de abajo, sustituyendo la clave de B.2 y la contraseña de A.3.

```

APP_NAME=WebLab
APP_ENV=production
APP_KEY=base64:PEGA_AQUI_LA_CLAVE_DE_B2
APP_DEBUG=false
APP_URL=https://webapp.cefppj.org.mx

APP_LOCALE=es
APP_FALLBACK_LOCALE=es
APP_FAKER_LOCALE=es_MX
APP_TIMEZONE=America/Mexico_City

LOG_CHANNEL=daily
LOG_LEVEL=error

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cefpotgn654_weblab
DB_USERNAME=cefpotgn654_TU_USUARIO
DB_PASSWORD=LA_CONTRASENA_DE_A3

SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_SECURE_COOKIE=true

QUEUE_CONNECTION=database
CACHE_STORE=database
FILESYSTEM_DISK=local

MAIL_MAILER=smtp
MAIL_HOST=mail.cefppj.org.mx
MAIL_PORT=587
MAIL_USERNAME=no-reply@cefppj.org.mx
MAIL_PASSWORD=
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=no-reply@cefppj.org.mx
MAIL_FROM_NAME=WebLab

VITE_APP_NAME=WebLab

```

Variable	Por qué ese valor
APP_ENV=production	No es decorativo: es lo que impide que el seeder cree el usuario admin con la contraseña password. Ver B.3.
APP_DEBUG=false	Con true, cualquier error muestra el stack trace y las variables de entorno al visitante. Innegociable en producción.
SESSION_SECURE_COOKIE=true	La cookie de sesión solo viaja por HTTPS. Requiere el Force HTTPS de A.5.
LOG_CHANNEL=daily	Rota los logs por día. Con single, laravel.log crece sin límite y se come la cuota.

Variable	Por qué ese valor
QUEUE_CONNECTION=databas e	No hay Redis ni un <i>worker</i> permanente; las colas viven en la base y las drena un cron (Parte F).
DB_HOST=127.0.0.1	MySQL es local a la cuenta. No expongas acceso remoto si no hace falta.

Correo

Deja MAIL_PASSWORD vacío hasta crear la cuenta no-reply@cefppj.org.mx en **cPanel** → **Email Accounts**. Si WebLab todavía no envía correos, puedes poner MAIL_MAILER=log y resolverlo después.

8. Parte E — Primer despliegue

- **cPanel** → **Git Version Control** → **Manage** sobre WebLab.
- Pestaña **Pull or Deploy**.
- Pulsa **Update from Remote** y espera a que termine.
- Pulsa **Deploy HEAD Commit**.

cPanel encola la tarea y ejecuta `.cpanel.yml`. El resultado queda en un log que puedes leer en **File Manager**:

```
/home/cefpotgn654/.cpanel/logs/
```

Comprobado: `.cpanel.yml` sí se ejecuta

Era la incógnita mayor de todo el plan, porque el administrador tiene el *shell access* deshabilitado y algunos hostings bloquean el despliegue junto con él. No es el caso: el primer despliegue real corrió y copió los archivos. **El Plan B de la sección 12 queda como reserva**, no como camino probable.

E.1 Qué debe haber pasado si todo salió bien

Ruta	Qué esperar
<code>~/weblab/</code>	El código completo, con <code>vendor/</code> y tu <code>.env</code> intacto.
<code>~/weblab/storage/</code>	Subcarpetas <code>framework/</code> y <code>logs/</code> con permisos 775.
<code>~/webapp.cefppj.org.mx/index.php</code>	El front controller que apunta a <code>../weblab</code> .
<code>~/webapp.cefppj.org.mx/build/</code>	Los <code>.js</code> y <code>.css</code> de Vite.
<code>~/webapp.cefppj.org.mx/storage</code>	Un enlace simbólico a <code>../weblab/storage/app/public</code> .
Base de datos	Las tablas de WebLab creadas en <code>cefpotgn654_weblab</code> (revisa en phpMyAdmin).

E.2 El motor de la base de datos: InnoDB, no MyISAM

El servidor creaba las tablas en MyISAM

Con el PHP ya corregido, el despliegue llegó a las migraciones y murió en la primera:

```
SQLSTATE[42000]: 1071 Specified key was too long; max key length is 1000 bytes
```

Ese límite de 1000 bytes es la firma de MyISAM. Un índice único sobre `varchar(255)` en `utf8mb4` ocupa 1020 bytes y no cabe.

La conexión de Laravel traía `'engine' => null`, que significa *usa el motor por defecto del servidor*. En desarrollo ese defecto es InnoDB y en este hosting es MyISAM: por eso las migraciones pasaban en local y fallaban aquí, que es el peor tipo de diferencia entre entornos.

El índice era solo el síntoma visible

Lo grave es que **MyISAM no soporta claves foráneas: las acepta en el SQL y las descarta sin avisar**. El esquema de WebLab usa `constrained()` en veinte migraciones. Si el índice hubiera cabido, la base se habría creado “bien” y habría quedado sin ninguna integridad referencial — un municipio podría apuntar a un estado inexistente, y nadie se enteraría hasta encontrar datos huérfanos meses después.

`config/database.php` ahora lo fija explícitamente en las conexiones `mysql` y `mariadb`:

```
'engine' => env('DB_ENGINE', 'InnoDB'),
```

Si te encuentras la base a medias

Una migración que falla a mitad deja tablas creadas pero sin registrar en `migrations`, y el siguiente intento muere con *“Table already exists”*. Se sale con un cron de un solo uso que rehace el esquema desde cero.

Borra todos los datos, así que solo vale mientras la base no tenga información real:

```
cd /home/cefpotgn654/weblab && /opt/alt/php83/usr/bin/php artisan migrate:fresh --force
```

E.3 Sembrar los catálogos que migrate no cubre

Sin este paso el sistema arranca a medias

`.cpanel.yml` corre `migrate --force` pero **no** `db:seed`. Parte de los catálogos viven en migraciones y llegan solos; el resto vive en *seeders* y se queda en cero.

Catálogo	Con solo migrate	De dónde viene
Estados	32	Migración
Municipios	2,478	Migración
Especies	7	Migración
Preguntas	5	Migración
Razas	0	Seeder
Pruebas	0	Seeder
Funciones zootécnicas	0	Seeder

E.4 El cron de un solo uso: seed y administrador

Sin shell, un cron es tu terminal. Crea **uno solo** que haga las dos cosas que faltan, déjalo correr una vez, lee el archivo y bórralo. En **cPanel** → **Advanced** → **Cron Jobs**, frecuencia *Once Per Five Minutes*:

```
/opt/alt/php83/usr/bin/php /home/cefpotgn654/weblab/artisan db:seed --force \
> /home/cefpotgn654/setup.log 2>&1 && \
/opt/alt/php83/usr/bin/php /home/cefpotgn654/weblab/artisan weblab:crear-admin \
dcastillo daniel@cefppj.org.mx --name="Daniel Castillo" \
>> /home/cefpotgn654/setup.log 2>&1
```

- Espera a que pase la hora programada y abre `/home/cefpotgn654/setup.log` en **File Manager**.
- Ahí estará la **contraseña generada** del administrador, en la línea *Contraseña generada:*. Cópiala.
- **Borra el cron y borra setup.log**. Mientras exista, la contraseña está en texto plano en el servidor.
- Entra al sistema y cámbiala desde el perfil.

Si prefieres elegir tú la contraseña

Añade `--password="..."` al comando, con un mínimo de 12 caracteres. A cambio, la contraseña queda escrita en el crontab en vez de en el log: en ambos casos hay que borrar el rastro después. El comando también sirve para reponer el acceso más adelante — si el usuario ya existe, actualiza su contraseña en lugar de duplicarlo.

9. Parte F — Tareas programadas

Laravel necesita dos procesos que en un servidor normal correrían como demonios. Aquí se resuelven con cron. Ve a **cPanel** → **Advanced** → **Cron Jobs**.

F.1 Scheduler

Frecuencia: **Once Per Minute** (* * * * *)

```
/opt/alt/php83/usr/bin/php /home/cefpotgn654/weblab/artisan schedule:run >/dev/null 2>&1
```

F.2 Procesador de colas

Frecuencia: **Once Per Five Minutes** (* /5 * * * *)

```
/opt/alt/php83/usr/bin/php /home/cefpotgn654/weblab/artisan queue:work \
--stop-when-empty --max-time=280 --tries=3 >/dev/null 2>&1
```

--stop-when-empty hace que el proceso termine al vaciar la cola, y --max-time=280 lo corta antes de que se solape con la siguiente ejecución. En hosting compartido, un queue:work sin límites acaba consumiendo la cuota de CPU y el proveedor lo mata.

F.3 Limpieza de logs

Frecuencia: **Once Per Week** (0 0 * * 0)

```
/usr/bin/find /home/cefpotgn654/weblab/storage/logs -name "*.log" \
-mtime +14 -delete >/dev/null 2>&1
```

Sobre el correo de cron

Arriba del formulario hay un campo **Cron Email**. Si lo dejas con una dirección real y quitas el `>/dev/null 2>&1`, recibirás un correo por cada ejecución: con el scheduler cada minuto, son 1,440 correos al día. Deja la redirección puesta, o vacía el campo.

10. Parte G — Verificación

#	Comprobación	Resultado esperado
1	Abrir https://webapp.cefppj.org.mx	Pantalla de login de WebLab con estilos aplicados
2	Candado del navegador	Certificado válido; la URL con <code>http://</code> redirige a HTTPS
3	Consola del navegador (F12)	Sin errores 404 de <code>/build/...</code>

#	Comprobación	Resultado esperado
4	Pedir <code>https://webapp.cefppj.org.mx/.env</code>	404 o 403. Si descarga algo, detén todo y revisa la Parte B.
5	Iniciar sesión	Entra al dashboard dentro del AuthenticatedLayout
6	Crear una Historia Clínica	Se guarda; los modales de propietario y dirección responden (son llamadas axios)
7	Navegar entre Catálogos	Cambian de página sin recarga completa (Inertia funcionando)
8	Revisar los otros 3 sitios	cefppj.org.mx, dms y siepesa siguen operativos tras el cambio de PHP
9	Leer <code>~/web lab/storage/logs/</code>	Sin excepciones nuevas

11. Parte H — Despliegues posteriores

Una vez que el circuito funciona, publicar un cambio es esto:

```
git push origin main          # GitHub Actions compila y actualiza la rama deploy
```

...y después, en cPanel, **Git Version Control** → **Manage** → **Update from Remote** → **Deploy HEAD Commit**.

H.1 Automatizarlo con el MCP del repositorio

El repositorio incluye un servidor MCP (`mcp/deploy/index.js`) que hace ese segundo paso por HTTPS, sin navegador. Crea `.env.deploy` en la raíz del proyecto, en tu Mac, a partir de `.env.deploy.example`:

```
CPANEL_HOST=webapp.cefppj.org.mx
CPANEL_PORT=2083
CPANEL_USER=cefpotgn654
CPANEL_TOKEN=el_token_creado_en_A4
CPANEL_REPO_ROOT=/home/cefpotgn654/repositories/WEBlab
CPANEL_APP_DIR=weblab
CPANEL_INSECURE_TLS=false

SOURCE_BRANCH=main
DEPLOY_BRANCH=deploy
GIT_REMOTE=origin
```

`.env.deploy` está en `.gitignore`: el token no debe subir al repositorio nunca.

Herramienta MCP	Qué hace
deploy	Espera a que GitHub Actions termine y dispara el despliegue en cPanel
deploy_status	Muestra el commit publicado frente al que está en el servidor
rollback	Vuelve a un build anterior de la rama deploy
remote_logs	Lee <code>storage/logs/laravel.log</code> del servidor
deploy_log	Lee el log del último despliegue de cPanel

Nunca hagas force-push a la rama deploy

cPanel mantiene un clon de esa rama y actualiza con `git pull`. Un *force-push* reescribe el historial y deja el clon del servidor en un estado del que solo se sale borrando y volviendo a clonar. La rama `deploy` es aditiva: cada build es un commit encima del anterior. Eso también es lo que hace posible el `rollback`.

12. Plan B: si .cpanel.yml no se ejecuta

Si el despliegue de Git Version Control resulta estar bloqueado junto con el shell, todavía hay dos caminos.

Opción B1 — Publicar con FTP o File Manager

Renuncias a la automatización, pero funciona seguro. Ya hay 6 cuentas FTP en la cuenta.

- En tu Mac: `composer install --no-dev --optimize-autoloader` y `npm run build`.
- Comprime todo menos `node_modules`, `.git`, `.env` y `storage`.
- Sube el ZIP a `/home/cefpotgn654/weblab` con File Manager y usa **Extract** (subir un ZIP y extraer es mucho más rápido que subir 20,000 archivos por FTP).
- Sube el contenido de `public/` a `/home/cefpotgn654/webapp.cefppj.org.mx`, y `deploy/cpanel-index.php` renombrado a `index.php`. Borra el `index.html` de relleno.
- Las migraciones se lanzan con un cron de un solo uso: créalo, deja que corra una vez, y bórralo.

```
/opt/alt/php83/usr/bin/php /home/cefpotgn654/weblab/artisan migrate --force \  
>/home/cefpotgn654/migrate.log 2>&1
```

Después lee `migrate.log` en File Manager para ver el resultado. El mismo truco sirve para `optimize`, `storage:link` o cualquier comando de artisan que necesites: el cron es tu terminal.

Opción B2 — Pedir SSH al hosting

Es la solución de fondo. El texto para el ticket:

```
Solicito habilitar acceso SSH (jailed shell) para la cuenta cefpotgn654.  
Necesito ejecutar Composer y comandos de Laravel artisan para desplegar  
una aplicación PHP 8.3 en el subdominio webapp.cefppj.org.mx.
```

```
Adicionalmente, solicito habilitar "site isolation" (CageFS + PHP Selector  
por dominio) para poder usar PHP 8.3 solo en ese subdominio, sin afectar  
a cefppj.org.mx, dms ni siepesa, que permanecerían en PHP 7.0.
```

Con SSH habilitado, todo este plan se simplifica: bastaría clonar `main`, correr `composer install` en el servidor y subir solo `public/build` compilado desde tu Mac.

13. Solución de problemas

Síntoma	Causa probable y solución
500 en todas las páginas, pero 404 correcto en rutas inexistentes	El caso más probable. <code>public_path()</code> apunta a <code>~/weblab/public</code> , que no existe, y <code>@vite</code> no encuentra el manifest. El front controller debe llamar a <code>usePublicPath(__DIR__)</code> . Ver sección 3.1.
Página en blanco de cPanel, o el <code>index.html</code> de relleno	No se borró <code>index.html</code> del document root. Apache lo sirve antes que <code>index.php</code> . Ver cambio 2 en B.1.
<i>500 Server Error</i> sin más detalle	Lee <code>~/weblab/storage/logs/</code> . Si está vacío, revisa cPanel → Metrics → Errors : suele ser permisos de storage o bootstrap/cache.
<i>Class not found / vendor/autoload.php missing</i>	El repositorio está en main, no en deploy. Ver C.4.
Composer detected issues... You are running 7.0.33	El PHP de la CLI no es el de la web. Usa <code>/opt/alt/php83/usr/bin/php</code> , no <code>/usr/local/bin/php</code> . Ver B.1.
<i>1071 Specified key was too long; max key length is 1000 bytes</i>	El servidor crea las tablas en MyISAM. Fija <code>'engine' => 'InnoDB'</code> en <code>config/database.php</code> . Ver E.2.
<i>1050 Table already exists al migrar</i>	Una migración anterior falló a medias. Rehaz el esquema con <code>migrate:fresh --force</code> . Ver E.2.
<i>The system cannot deploy tras un fallo</i>	Artisan dejó un <code>error_log</code> sin rastrear en el repositorio del servidor. Bórralo. Ver B.1.
El clonado falla o pide credenciales	WEBlab es privado. cPanel no acepta credenciales en la URL: hay que dejarlas en <code>.git-credentials</code> . Ver C.2.
Los catálogos de razas y pruebas salen vacíos	Faltó el <code>db:seed</code> : esos viven en seeders, no en migraciones. Ver E.2 y E.3.
Existe un usuario admin que no creaste	El <code>.env</code> no tiene <code>APP_ENV=production</code> . Bórralo, corrige el <code>.env</code> y vuelve a desplegar.
Estilos rotos, la página sale sin formato	Falta <code>build/</code> en el document root, o <code>APP_URL</code> no coincide con el dominio real.
404 en todas las rutas menos la raíz	Falta el <code>.htaccess</code> de Laravel en el document root. Debe llegar con el rsync de <code>public/</code> .
<i>SQLSTATE[HY000] [1045] Access denied</i>	Credenciales del <code>.env</code> . Recuerda el prefijo <code>cefpotgn654_</code> en base y usuario, y que el usuario esté asignado a la base con <code>ALL PRIVILEGES</code> .
<i>No application encryption key</i>	Falta <code>APP_KEY</code> en el <code>.env</code> , o se pegó sin el prefijo <code>base64:</code> .
Sesión que se pierde al navegar	<code>SESSION_SECURE_COOKIE=true</code> sin Force HTTPS activo. Ver A.5.

Síntoma	Causa probable y solución
Cambié el .env y no surte efecto	La configuración quedó cacheada por <code>artisan optimize</code> . Vuelve a desplegar, o corre <code>optimize:clear</code> con el truco del cron de un solo uso (B1).
Los otros sitios se cayeron	El cambio de PHP de A.2. Vuelve a 7.0 en Select PHP Version y aplica una alternativa de A.1.
phpMyAdmin da Access denied	Sesión de cPanel caducada (me pasó durante la revisión). Cierra sesión, vuelve a entrar y ábrelo de nuevo.

14. Lista de verificación

Parte	Tarea	Hecho
A	HECHO - PHP de la web en 8.3.33	OK
A	HECHO - Base cefpotgn654_weblab creada y con usuario	OK
B	HECHO - .cpanel.yml: subdominio, index.html y PHP 8.3 real	OK
B	HECHO - Front controller corrige public_path	OK
B	HECHO - config/database.php fuerza InnoDB	OK
B	HECHO - Administrador de prueba bloqueado en produccion	OK
C	HECHO - Token de GitHub y .git-credentials	OK
C	HECHO - Repositorio clonado en la rama deploy	OK
D	HECHO - ~/weblab/.env con APP_ENV=production	OK
E	HECHO - Deploy HEAD Commit completado	OK
E	HECHO - Esquema migrado en InnoDB y catalogos sembrados	OK
E	HECHO - Administrador creado	OK
F	HECHO - Los tres crons permanentes, con el PHP correcto	OK
G	HECHO - /login responde 200 y .env responde 403	OK
E	Contraseña del admin cambiada y setup.log borrado	
A	Force HTTPS activado en webapp.cefppj.org.mx	
A	cefppj.org.mx, dms y siepesa verificados tras el cambio de PHP	
G	Recorrido funcional: crear una historia clinica de prueba	

Parte	Tarea	Hecho
H	.env.deploy configurado en la Mac para desplegar desde Claude Code	

Lo que queda. Nada estructural: cambiar la contraseña del administrador y borrar `setup.log`, encender Force HTTPS, comprobar que los otros tres sitios siguen bien tras el cambio de PHP, y hacer un recorrido funcional creando una historia clínica de prueba.

Y a partir de aquí, publicar un cambio son cinco minutos: `git push origin main`, esperar a GitHub Actions, y en cPanel *Update from Remote* seguido de *Deploy HEAD Commit*. O una sola llamada al MCP si configuras `.env.deploy` (Parte H).

Si algo se rompe en un despliegue futuro

Mira primero `~/.cpanel/logs/`: el log del último `vc*_git_deploy.log` lleva la salida de cada tarea de `.cpanel.yml`, con su código de salida. Ahí aparecieron los tres fallos de este despliegue, uno detrás de otro. Si el botón de desplegar se queda deshabilitado, es que quedó algo sin rastrear en el repositorio del servidor — casi siempre un `error_log`.